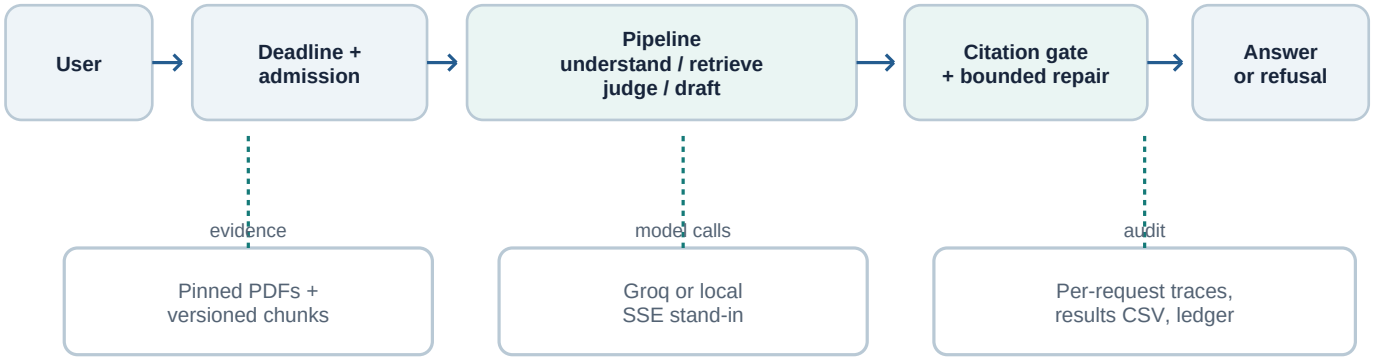
The Relay

Additional architecture, measurement, and decision detail supporting the two-page summary. This appendix preserves design rationale and limits without changing the generated results tables.

1. System architecture

The system is a deadline-aware, evidence-grounded pipeline rather than a direct chat wrapper. Every model answer is constrained by passages from the three required, version-pinned papers: Attention Is All You Need (arXiv:1706.03762v7), Moshi (arXiv:2410.00037v1), and Dynamic Chunking (arXiv:2507.07955v2).



Request path

- **Receive and admit.** The request receives a request ID, scheduled arrival, deadline, priority, and patience window. A shared limiter checks the latest known model-specific RPM, TPM, RPD, and TPD buckets before a provider call.
- **Understand and retrieve.** The pipeline rewrites the user question, then searches local, versioned PDF chunks. The retrieved chunk IDs and PDF page metadata travel with the request.
- **Judge, draft, and gate.** The model judges whether the retrieved passages support an answer and drafts it. A deterministic sentence gate verifies each citation marker points to a retrieved chunk and checks cited text for numbers, quoted phrases, and named terms. Accepted markers render to a human-readable paper/version/PDF-page citation.
- **Repair or refuse.** The full path allows one bounded repair after a gate failure. The lite path skips repair. If support still fails, the pipeline returns a safe refusal instead of unsupported text. A retrieval-only extractive path exists for tight budgets.

The gate is a safety check over citation membership and selected literal evidence. It does not prove that every semantic inference is correct; human grading remains necessary.

2. Contracts that make results auditable

The project deliberately separates the compact assignment CSV from richer traces. Request ID is the join key, so quality judgments and latency details can be reviewed without expanding the required CSV schema.

results.csv contract

The required columns remain exactly: request_id, scenario, source, start_time, first_token_ms, total_ms, status, retries, tokens_in, tokens_out, retrieval_setting, answer_grade. Scheduled arrival is the start time. first_token_ms is the first character of the answer delivered after the gate; drafts rejected by the gate do not count as user-visible output. A failed or rate-limited request can therefore have elapsed time but no delivered first token.

Per-request trace

- Stores the question, final answer, optional question ID, retrieved chunk IDs, scenario, arrival and deadline, degradation level, cache/coalescing flags, and whether the user gave up.
- Stores one span per wait, work, or retry operation, including attempt, model, status, provider TTFT, and input/output, cached, and reasoning token counts when present.
- Uses create-only trace writes. Decision 0004 added these fields to make human-graded answers reviewable while leaving the assignment CSV unchanged. The assistant proposed the trace-audit option; the user accepted it after considering the added retention and audit value.

Provider ledger and tables

The provider ledger records the experiment, model, token usage, status, and a snapshot of x-ratelimit headers. Real calls use a per-experiment approval permit tied to the estimated request and token budget. analysis/make_tables.py computes result tables from results.csv and supporting evidence; analysis/checks.py compares the generated block in WRITEUP.md with a fresh render. Percentiles use nearest rank, report their sample size, and show p99 only at n >= 100.

These contracts are why the 134 answer grades were copied only into matching successful E5 answer_grade cells under accepted decision 0005; the grade file remains the source record, and the seven rate-limited attempts remain ungraded. The pre-merge snapshot is preserved.

3. Retrieval design and evaluation

PDF ingestion uses PyMuPDF to preserve 1-based PDF page locations and emits three chunk configurations. Retrieval is local BM25 by default. The code also supports CPU sentence-transformer embeddings with reciprocal-rank fusion and an optional cross-encoder reranker.

- **R1:** 900-character pieces, 90-character overlap, top 3, no reranker.
- **R2:** 1,600-character pieces, 160-character overlap, top 4, no reranker.
- **R3:** 2,600-character pieces, 260-character overlap, top 6, cross-encoder reranking.

The paid comparison reused the same 15 questions across three candidate Groq models and R1/R2/R3. Semantic embeddings were explicitly disabled for this comparison; only R3 used a reranker. There are 134 human grades because seven attempts were rate-limited. R3 had the highest correct-and-cited rate, but more tokens and longer first-token time; the majority of graded outputs remained refusals. The full per-setting metrics are in Table 3 of WRITEUP.md and are generated, not manually transcribed here.

The naive BM25 examples demonstrate cross-paper lexical collisions: a Moshi GPU question retrieves a Transformer passage; a Moshi BLEU question finds the Transformer paper’s BLEU result; and a Transformer response-time question returns Moshi’s 160 ms theoretical latency. These are retrieval errors, not claims about the source papers.

4. Scheduling, caching, and failure policy

Admission and deadlines

A bounded priority queue drops expired work and, when full, sheds the least urgent queued request. A separate degradation policy selects the richest path whose estimated latency fits the remaining deadline: full, lite, extractive, then optionally wait; work is dropped when the deadline is gone or no allowed path fits. The configured queue capacity, concurrency ceiling, patience, and latency estimates are policy hypotheses, not provider measurements.

Cache and duplicate traffic

The load generator samples popular questions with Zipf-like skew, so duplicates can arrive concurrently. AnswerCache combines TTL entries with single-flight tasks. The canonical hash key includes question, provider, model map, retrieval setting, semantic/rerank flags, and degradation level. Only successful, answerable, gate-accepted results are cached. This avoids replaying rejected drafts or reusing a result across a different model or retrieval configuration. TTL and capacity defaults remain unverified workload settings.

Retries

Retries are permitted only for timeout, rate-limit, and server-error classes. The policy honors Retry-After when present; otherwise it uses full-jitter exponential backoff. It refuses a retry after the per-request attempt cap, expired deadline, or exhausted process-wide retry budget. These defaults are labeled unverified. The answer-quality experiment disabled retries so a rate limit was recorded instead of replaying a request. A separate seeded stand-in test returned a persistent 503: retrying added 351.2 ms and still failed, while the no-retry case ended in 2.203 ms.

What the controlled stand-in does

A FastAPI server implements an OpenAI-compatible streaming endpoint, Groq-shaped rate-limit headers, rolling request/token windows, configurable seeded slow successes, 5xxs, hangs, SSE chunking, and load-dependent delay. Admin endpoints allow limit changes during a run. The load generator is open-loop and includes steady, burst, sustained overload, halved per-minute limit, exhausted daily budget, and retry-worse comparisons.

Source trail: WRITEUP.md; results/results.csv; budget_ledger.csv; results/raw/; results/traces/; docs/decisions/0001-0005; relay/pipeline.py, gate.py, limiter.py, scheduler.py, retry.py, cache.py, retrieval.py, llm_client.py; standin/server.py and standin/config.yaml. Generated tables remain in WRITEUP.md; this appendix does not replace them.

5. What the real measurements support

Observed provider evidence

Successful real responses exposed 8,000 tokens per minute and 1,000 requests per day in x-ratelimit headers. Across 332 saved header observations with a positive request deficit and parseable reset, reset spacing agreed within 0.01 seconds of 86.4 seconds per spent request, equivalent to a 24-hour refill at 1,000 requests. A real HTTP 429 was observed with Retry-After 16 seconds; its response body was not captured. The account limits page was not opened, so the published 30 RPM and 200,000 TPD figures remain documentation references, not confirmed account measurements.

Five real steady requests used openai/gpt-oss-20b. Their first-token measurements ranged from 1,330.4 to 3,174.9 ms, with p50 1,517.7 ms. This sample is too small to fit a latency distribution or report p99. The 15-question comparison retained three model candidates—openai/gpt-oss-120b, openai/gpt-oss-20b, and qwen/qwen3.8-27b—and did not claim that tiny OK probes selected a quality winner or a final per-step model map.

Stand-in calibration boundary

Published stand-in runs used zero base delay, zero added in-flight delay, no probabilistic slow/failure/hang injection, 60 RPM, 30,000 TPM, 1,000 RPD, and a 1,000,000 TPD placeholder. Only the RPD figure matched an observed header; the TPM header was 8,000. The five real steady samples were not enough to estimate latency parameters. Stand-in steady p50 was 3.8 ms (n=11), far below the real steady p50 of 1,517.7 ms (n=5); therefore stand-in burst/overload timing represents local queueing and policy behavior, not a calibrated Groq latency model.

Quality and cost interpretation

Across R1/R2/R3, 134 delivered answers were graded: 11 correct and cited, 27 correctly refused, and 96 incorrectly refused answerable questions. Seven E5 attempts were rate-limited and have no answer grade. Published-price cost is a hypothetical comparison; the free-tier experiment did not produce a bill. The grade, timing, token, and cost sample sizes/coverage are shown in generated tables.

Known unknowns

- Real daily exhaustion was not attempted because the last observed headers lacked a daily-token remaining value and other activity on the account was unknown.
- The stand-in's zero added delay is the least representative parameter. The latency fit (E1) and wait-until-daily-recovery behavior were deliberately omitted; S6 was undefined in both brief and plan.
- The configured retry, queue, patience, cache, and degradation thresholds remain unverified defaults unless a specific run calibrated them.

6. Decision ownership and what to defend

The user's documented engineering direction set four competing objectives—quality, latency, provider limits, and cost—and asked for comparable experiments rather than intuition alone. The user required budget estimates before live calls, kept 429s visible, separated provider TTFT from delivered first-token time, and required tables to be reproducible from request IDs and results.csv. The user also identified the missing answer_grade values as a reviewer reproducibility gap.

Decision provenance matters. The assistant proposed adding question/answer/evidence data to traces; the user accepted decision 0004 because auditability justified the retention cost. In the architecture conversation, the user corrected the assistant's second-key suggestion: one account only, never rewrite a 429 as success, and retry only under retry-after/jitter/deadline/global-budget controls. The user also rejected unsupported p99 claims and required nearest-rank sample-size rules.

The defensible story is not that every first idea was right. It is that the user made hypotheses, accepted correction when measurements or contracts exposed a flaw, set the final safety and reproducibility constraints, and chose real R2/R3 comparisons after limits reset while preserving failures. LLM support provided direction; the user states the system requirements and write-up are theirs. The transcript README records that some dialogue is reframed rather than verbatim.